Telco Churn

MSITM 6341 - Group A midterm project

- 1. Dataset
- 2. Cleaning and Preprocessing
- 3. Model Results
- 4. Predict a Customer

Data: IBM Telco Customer Churn (Kaggle). Models: Logistic Regression and Random Forest (scikit-learn).

Telco Customer Churn

Source: Kaggle - Telco Customer Churn (BlastChar), an IBM sample dataset of a fictional telecom company. Each row is one customer; **Churn** says whether they left in the last month.

Customers (rows)	Columns	Churn rate	Missing values found
7,043	21	26.5%	11

Variables

Built from the cleaned data, so TotalCharges is numeric and SeniorCitizen reads Yes/No.

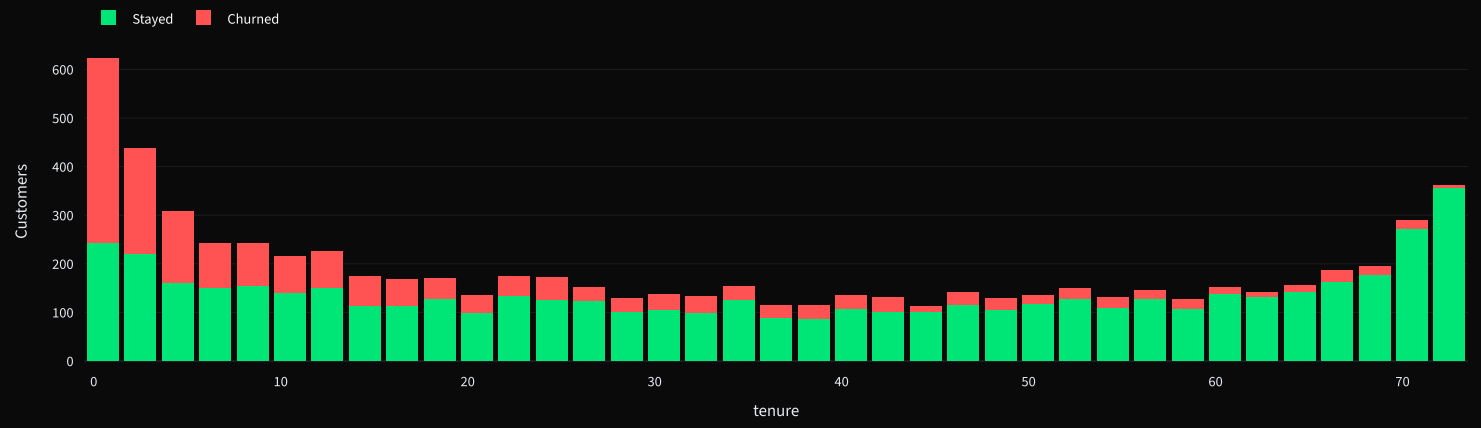
Variable	Type	Range / categories	Description
customerID	Categorical	7,043 unique values	Unique customer identifier (not used for modeling)
gender	Categorical	Female, Male	Customer gender
SeniorCitizen	Categorical	No, Yes	Whether the customer is 65 or older
Partner	Categorical	No, Yes	Whether the customer has a partner
Dependents	Categorical	No, Yes	Whether the customer has dependents (children, etc.)
tenure	Numeric	0.00 to 72.00	Number of months the customer has been with the company
PhoneService	Categorical	No, Yes	Has a phone line
MultipleLines	Categorical	No, Yes	Has more than one phone line
InternetService	Categorical	DSL, Fiber optic, No	Type of internet connection
OnlineSecurity	Categorical	No, Yes	Subscribes to the online security add-on

Explore a variable

Variable

tenure

Distribution of tenure by churn outcome



Exploratory Data Analysis (Task 2.2)

🛠 The exploratory data analysis and visualizations from Task 2.2 will be added here by our teammates.

Data: IBM Telco Customer Churn (Kaggle). Models: Logistic Regression and Random Forest (scikit-learn).

Cleaning and Preprocessing

Every step below comes from `src/task_2_1_clean_preprocess.py`. The decision notes are read directly from the code's docstrings.

Before and after

Raw data	Cleaned data	Model-ready (train + test)
7,043 x 21	7,043 x 21	5,634 + 1,409 x 28

Cleaning

1. Strip whitespace

`strip_whitespace()` - b1. Strip leading/trailing spaces from every text column.

2. Fix TotalCharges type and impute blanks

`handle_missing_values()` - b3. Report missing values and impute blank TotalCharges with 0.

Blank TotalCharges	Of those with tenure 0	Imputed value
11	11 of 11	$0

> **DECISION:** impute TotalCharges = 0 rather than drop or use the mean/median. Every one of these customers has tenure == 0: they just signed up and have not been billed yet, so their true total really is $0. Imputing 0 records that true value and keeps all 7,043 customers. Dropping 11 rows (0.16%) would also be acceptable but discards valid customers, and mean/median imputation would wrongly give brand-new customers thousands of dollars in charges.

3. Remove duplicates

`remove_duplicates()` - b4. Report and remove duplicate rows and duplicate customer IDs.

Duplicate rows	Duplicate customer IDs
0	0

4. Check charges consistency

`check_charges_consistency()` - b5b. Flag customers whose TotalCharges doesn't match tenure x MonthlyCharges.

5. Standardize categories

`standardize_categories()` - b6. Make category labels consistent and remove redundant values.

"No internet service" → "No"	"No phone service" → "No"	SeniorCitizen

9,156 values 682 values 0/1 → No/Yes

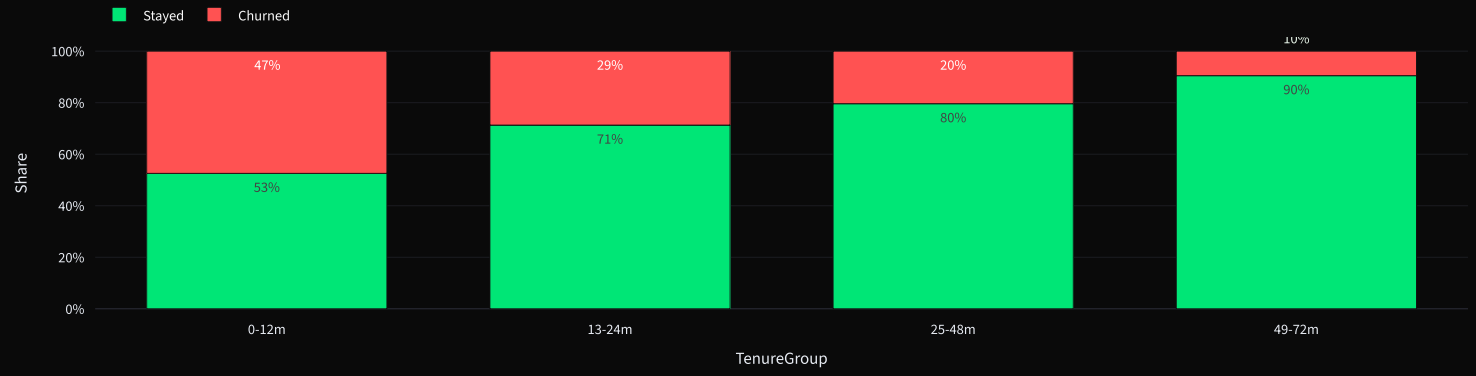
> **DECISION:** recode SeniorCitizen from 0/1 to "Yes"/"No". Every other yes/no column uses text labels, so this keeps the cleaned EDA file consistent and makes charts readable. It is encoded back to 0/1 in preprocessing.

> **DECISION:** recode "No internet service" (six add-on columns) and "No phone service" (MultipleLines) to "No". That information is already captured by the InternetService and PhoneService columns, so keeping it would only create redundant dummy columns later. "No" is accurate: the customer does not have that add-on.

Engineered features

- **NumServices:** how many add-on services the customer pays for (0-8). Customers who are more "embedded" in the product may be less likely to leave.
- **AvgMonthlySpend:** TotalCharges / tenure, i.e. what they actually paid per month on average. Differs from MonthlyCharges when prices or plans have changed. For tenure 0, fall back to the current MonthlyCharges (avoids divide-by-zero).
- **TenureGroup:** bucket tenure into customer-lifecycle stages. Churn risk is usually highest in the first year, so a group label can capture that pattern more directly than a raw month count. DECISION: bins of 0-12, 13-24, 25-48 and 49-72 months. The first two years get their own groups because that is where churn behavior changes fastest.

Churn rate by tenure group: new customers leave most



Encoding, splitting, and scaling

One-hot encoding

`encode_one_hot()` - Technique 2b: one-hot encode multi-category nominal columns.

> **DECISION:** one-hot rather than label encoding. These categories have no natural order ("Electronic check" is not "greater than" "Mailed check"), so label encoding them as 0/1/2/3 would invent a false ranking.

> **DECISION:** drop_first=True removes one dummy per column to avoid perfect multicollinearity, which matters for Logistic Regression.

Stratified 80/20 split

`split_data()` - Split into stratified 80/20 train and test sets.

> **DECISION:** split before scaling. The scaler must learn its mean/std from the training data only. If it were fit on the full dataset, information from the test set would leak into training and make model scores look better than they really are.

> **DECISION:** 80/20 split with stratify=y, which keeps the same churn ratio (~26.5%) in both sets. This matters because the classes are imbalanced.

Standard scaling

`scale_features()` - Technique 3: standardize continuous columns (fit on train only).

> **DECISION:** scale only the continuous columns and leave the 0/1 encoded columns as they are, so they stay easy to interpret.

Data: IBM Telco Customer Churn (Kaggle). Models: Logistic Regression and Random Forest (scikit-learn).

Model Results

Two algorithms, **Logistic Regression** and **Random Forest**, both with `class_weight="balanced"` because only 26.5% of customers churn. Tuned with 5-fold `GridSearchCV` on the training set (scoring = F1). All numbers below are on the **held-out test set** (1,409 customers).

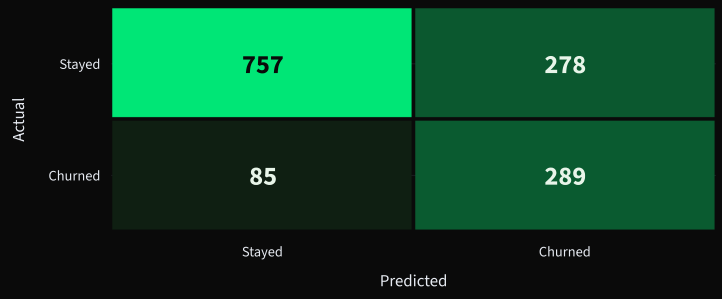
Comparison

Model	Accuracy	Precision	Recall	F1	ROC AUC
Baseline: predict everyone stays	0.735	0.000	0.000	0.000	0.500
Baseline Logistic Regression	0.733	0.498	**0.791**	0.612	**0.842**
Baseline Random Forest	**0.766**	**0.551**	0.636	0.591	0.826
Tuned Logistic Regression	0.742	0.510	0.773	0.614	0.839
Tuned Random Forest	0.757	0.528	0.773	**0.628**	**0.842**

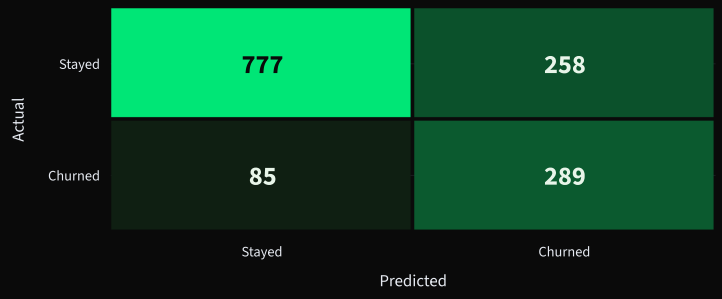
Green = best value in each column. The 'everyone stays' baseline is 73.5% accurate but catches no churners, which is why we judge the models on recall and F1, not accuracy.
Best parameters - Logistic Regression: {'C': 0.01}; Random Forest: {'max_depth': 10, 'n_estimators': 300}

Confusion matrices (threshold 0.5)

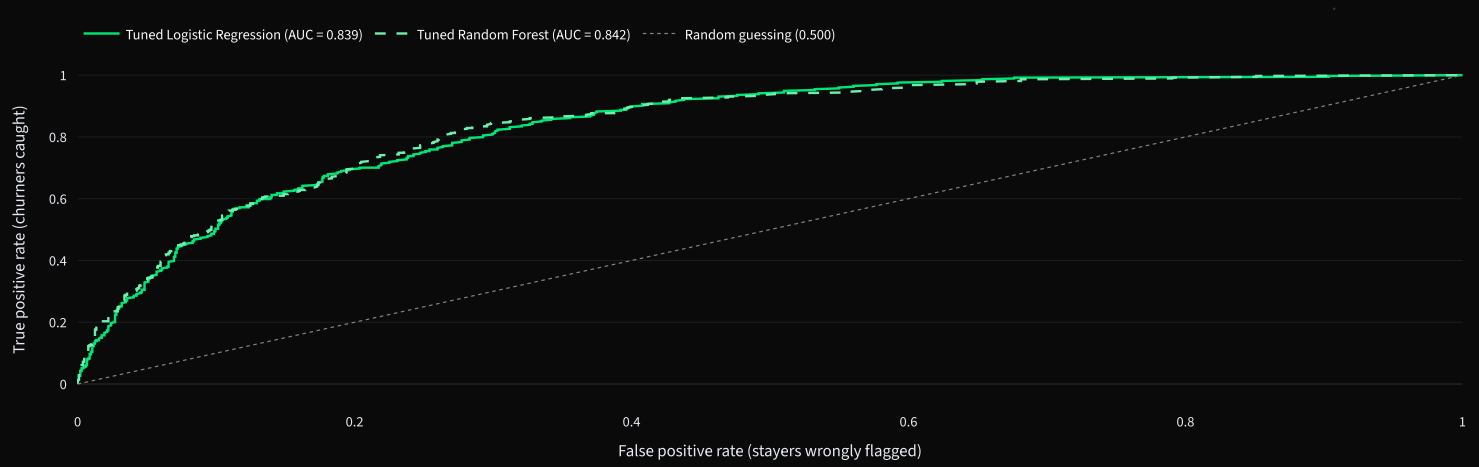
Tuned Logistic Regression



Tuned Random Forest



ROC curves



Choose the decision threshold

A model outputs a churn **probability**; the threshold decides when that counts as *will churn*. **Lower** the threshold and you catch more real churners (recall goes up), but you also flag more loyal customers by mistake (precision goes down). **Raise** it and the opposite happens. The right setting depends on what costs more: losing a customer, or sending a retention offer to someone who would have stayed.

Model

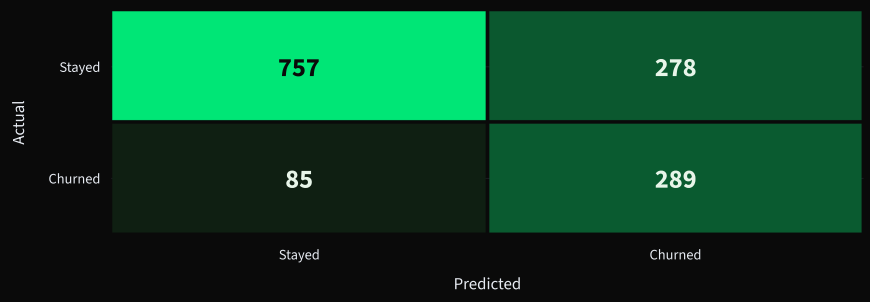
Tuned Logistic Regression

Threshold

0.50

Recall	Precision	Accuracy	F1
77.3%	51.0%	74.2%	61.4%

Tuned Logistic Regression at threshold 0.50



In plain English

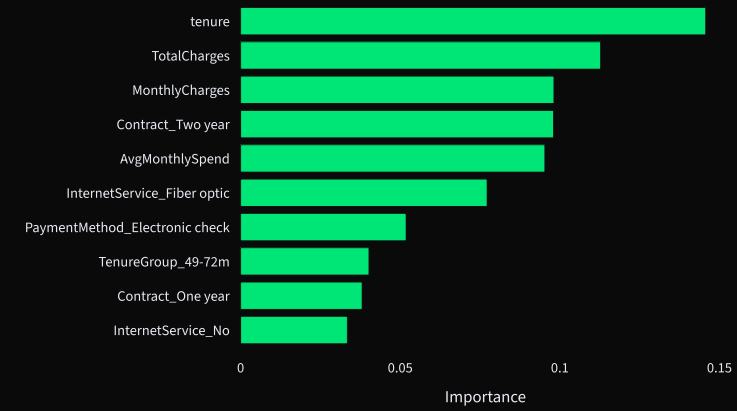
- Catches **289 of 374** customers who really left.
- Misses **85** churners (they leave without warning).
- Flags **278** customers who would have stayed (false alarms that would receive a retention offer).

What drives churn? Top 10 features

Logistic Regression coefficients
red = pushes toward Churned, green = pushes toward Stayed



Random Forest feature importances
how much each feature matters (no direction)



Telco Churn

MSITM 6341 - Group A midterm project

- 1. Dataset
- 2. Cleaning and Preprocessing
- 3. Model Results
- 4. Predict a Customer

Data: IBM Telco Customer Churn (Kaggle). Models: Logistic Regression and Random Forest (scikit-learn).

Predict a Customer

Fill in a customer profile, or load an example, then press **Predict**.

Random customer | High risk example | Low risk example

Customer

Gender: Male

Senior citizen (65+): No

Has partner: Yes

Has dependents: Yes

Services

Phone service: Yes

Multiple lines: Yes

Internet service: DSL

Online security: Yes

Online backup: Yes

Device protection: Yes

Tech support: Yes

Streaming TV: No

Streaming movies: No

Account

Tenure (months): 60

Monthly charges (USD): 65.00

Contract: Two year

Paperless billing: No

Payment method: Credit card (automatic)

Total charges (tenure x monthly)

$3,900.00

Predict

Prediction

Logistic Regression

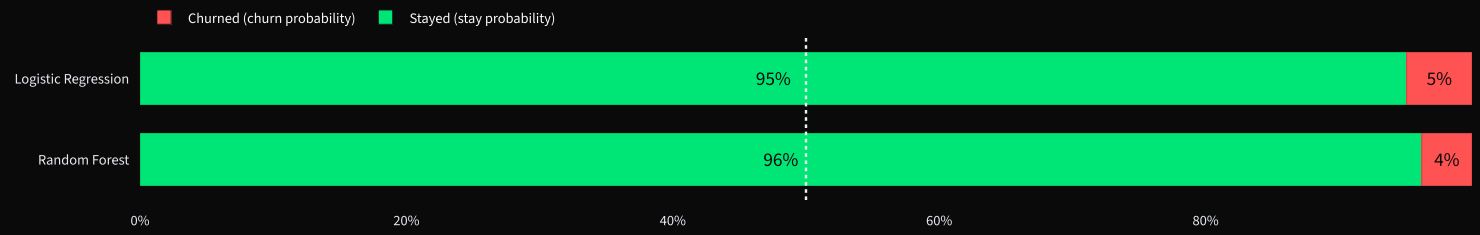
Stayed

↑ 5% churn risk

Random Forest

Stayed

↑ 4% churn risk



These are risk scores, not calibrated real-world probabilities: class_weight="balanced" makes the models treat churners as if they were as common as stayers, which pushes scores upward. Use them to rank customers by risk; the label uses the default 0.5 threshold (dotted line).